INDIAN INSTITUTE OF TECHNOLOGY ROORKEE

Distributed Systems

Global State

Recap

NTP

Global state

Notations:

- LS_i : local state of process i
- send(m_{ij}) : send event of message m_{ij} from process i to process j
- rec(m_{ij}) : receive event of message
- time(x) : time at which state x was recorded
- time (send(m)) : time at which send(m) occured

- send(m_{ij}) ϵ LS_i iff

  time(send(m_{ij})) < time(LS_i)

- rec(m_{ij}) ϵ LS_j iff

  time(rec(m_{ij})) < time(LS_j)

- **transit**(LS_i, LS_j) = { m_{ij} | send(m_{ij}) ϵ LS_i ∧ rec(m_{ij}) ∉ LS_j}

- **inconsistent**(LS_i, LS_j) =

  {m_{ij} | send(m_{ij}) ∉ LS_i ∧ rec(m_{ij}) ϵ LS_j}

Global state: collection of local states

GS = {LS_1, LS_2,..., LS_n}

GS is consistent iff

for all i, j, $1 \le i, j \le n$,

inconsistent(LS_i, LS_j) = Φ

inconsistent(LS_1, LS_2)
" (LS_3, LS_7)
" (LS_1, LS_3)

GS is transitless iff

for all i, j, $1 \le i, j \le n$,

transit(LS_i, LS_j) = Φ

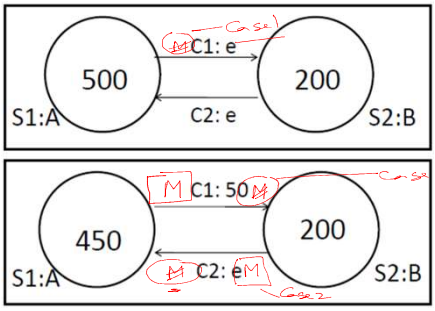
GS is strongly consistent if it is consistent and transitless

- corresponds to a consistent global state in which all channels are empty

Chandy-Lamport's Algorithm

- Uses special marker messages.
- Communication channels are assumed to be FIFO
- One process acts as initiator, starts the state collection by following the marker sending rule below.
- Marker sending rule for process P:
  - P records its state
  - for each outgoing channel C from P on which a marker has not been sent already, P sends a marker along C before any further message is sent on C

- When Q receives a marker along a channel C:
  - If Q has not recorded its state then Q records the state of C as empty; Q then follows the marker sending rule
  - If Q has already recorded its state, it records the state of C as the sequence of messages received along C after Q's state was recorded and before Q received the marker along C
- Each initiation needs its own unique marker
- Each process may finally send the recorded information to the initiator

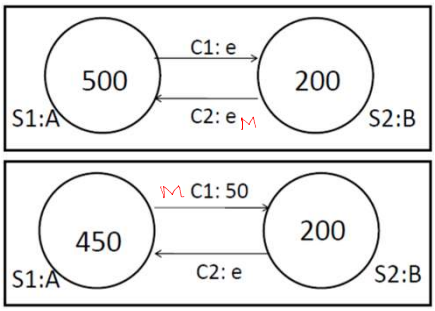


Case 1: A starts. Records state and then sends marker followed by the message on C1

Case 2: A starts. Sends message on C1. Records state and sends marker on C1.

Case 1 A: <500> B: <200> C1: < > C2: < >

Case 2: A: <450> B: <250> C1: < > C2: < >



Case 3: B starts. Records state and then sends marker on c2. A sends message and then receives marker.

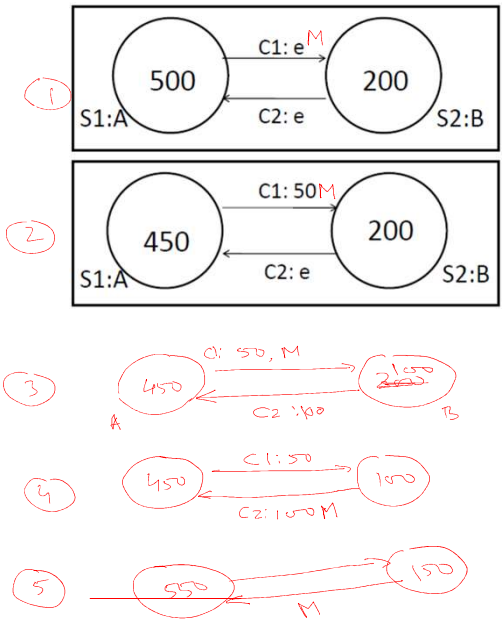
Case 3: B: <200> A: <450> C1: <50> C2: < >

LS_A

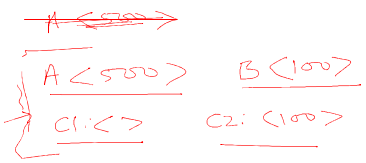
LS_B

Chandy-Lamport's Algorithm (Points to Note)

- Markers sent on a channel distinguish messages sent on the channel before the sender recorded its states and the messages sent after the sender recorded its state
- The state collected may not be any state that actually happened in reality, rather a state that "could have" happened

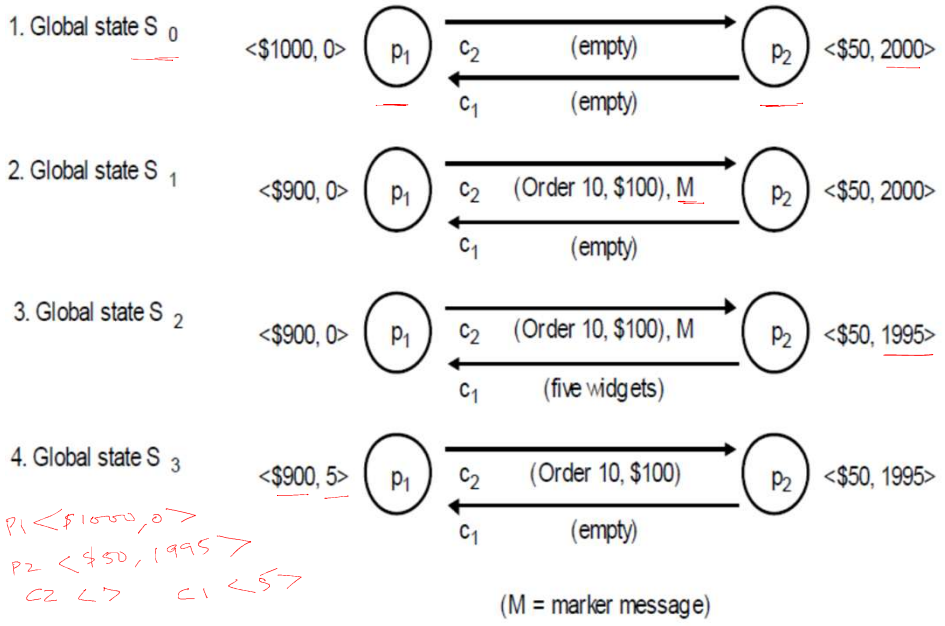


Case 4: A records state and then sends marker followed by the message on c1. B sends 100 on c2 and then receives marker.



Points to Note:

- Network should be strongly connected (works for connected, undirected also)
- Global state recording algorithm can be initiated by several processes concurrently with each process getting its own version of a consistent global state.
- Marker may be <process-id, seq-number>
- Message complexity O(|E|), where E = no. of links



Ref: Distributed Systems by A S Tanenbaum and M V Steen

Cuts of a Distributed Computation

- A cut of a distributed computation is a set

  $C = \{c_1, c_2, \ldots, c_n\}$

  where c_i is the cut event at site i

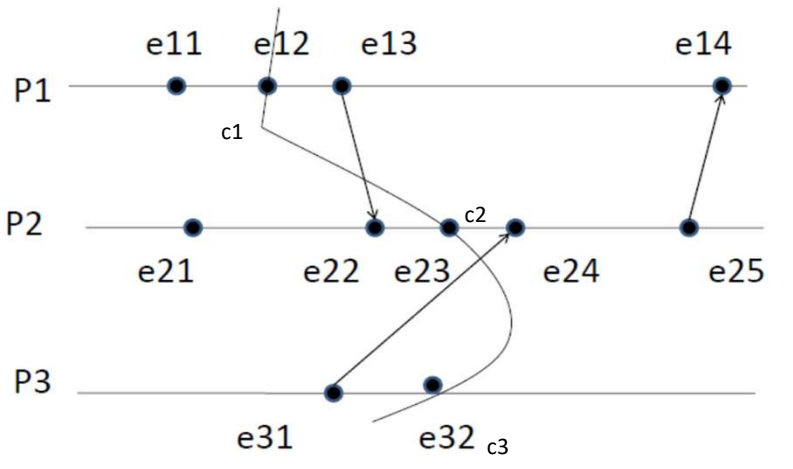
- A cut C is a consistent cut iff for all i, j there does not exists an e_i, e_j such that

  $(e_i \rightarrow e_j)$ and $(e_j \rightarrow c_j)$ and $(e_i \not\rightarrow c_i)$

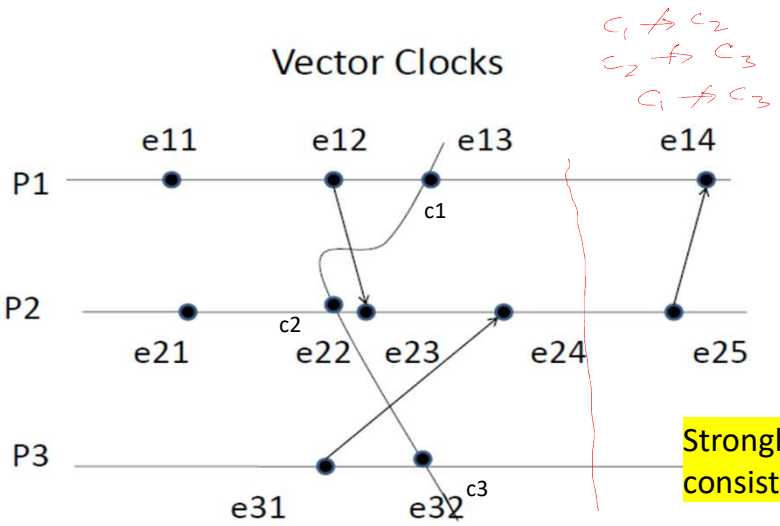
- **Theorem:** A cut C is consistent if and only if no two cut events are causally related

  for all c_i, c_j :: $\neg(c_i \rightarrow c_j)$ and $\neg(c_j \rightarrow c_i)$

Vector Clocks



$(e_i \rightarrow e_j)$ and $(e_j \rightarrow c_j)$ and $(e_i \not\rightarrow c_i)$



Strongly consistent?

$(e_i \rightarrow e_j)$ and $(e_j \rightarrow c_j)$ and $(e_i \not\rightarrow c_i)$

Time of a Cut

- Let VTc_i is the vector timestamp assigned to cut event c_i at site i
- Vector Time of the cut

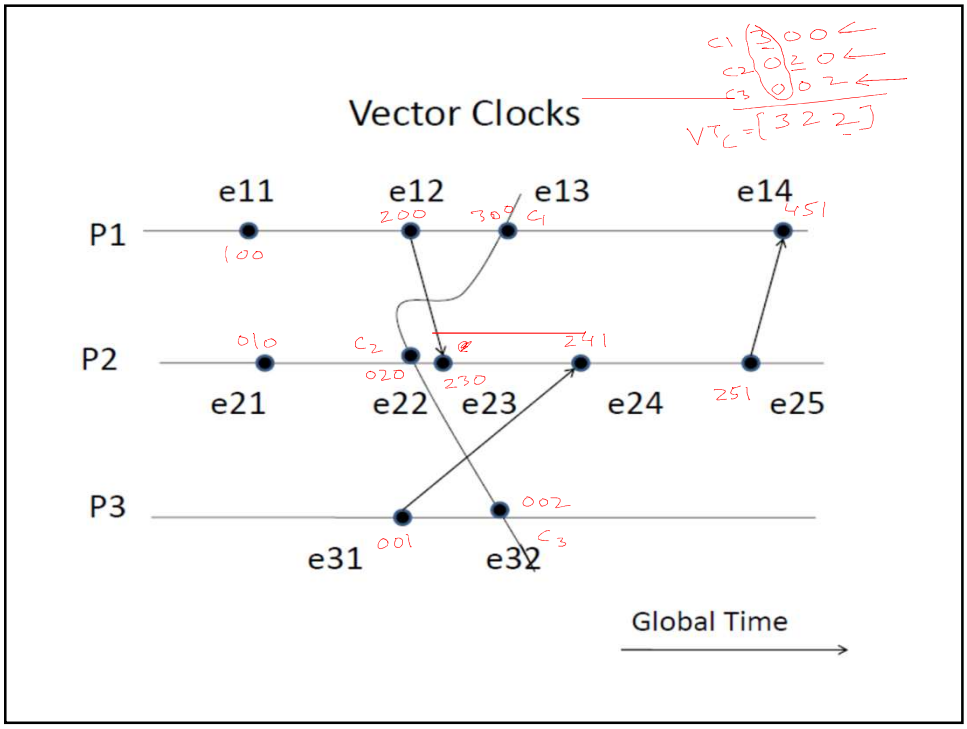
  $VTc = sup(VTc_1, VTc_2, ..., VTc_n)$

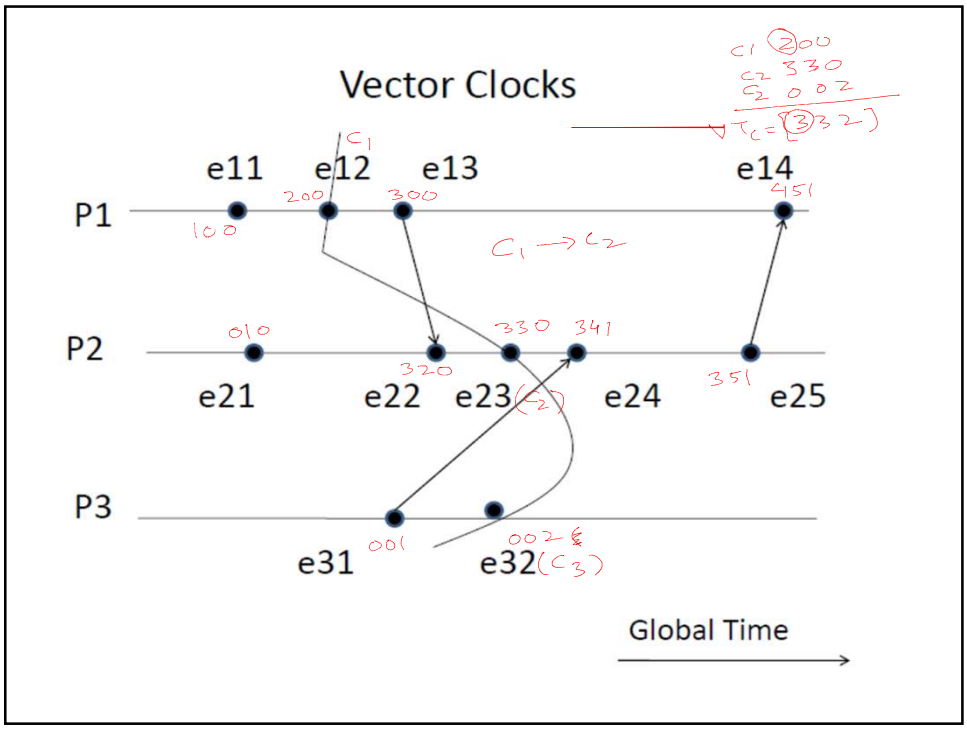
- sup is a component wise maximum operation

  $VTc[i] = max(VTc_1[i], VTc_2[i], ..., VTc_n[i])$

- **Theorem:** If C is a cut with vector times VTc, then the cut is consistent if and only if

  $VTc = (VTc_1[1], VTc_2[2], ..., VTc_n[n])$


Vector Clocks
e11
e12
e13
e14
P1
100
200
300
451
P2
010
020
230
241
251
e21
e22
e23
e24
e25
P3
001
002
e31
e32
Global Time


Vector Clocks
e11
e12
e13
e14
P1
100
200
300
451
P2
010
320
330
341
351
e21
e22
e23
e24
e25
P3
001
002
e31
e32
Global Time